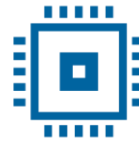




Transilvania
University
of Brasov



Transilvania
University
of Brasov

FACULTY OF ELECTRICAL ENGINEERING
AND COMPUTER SCIENCE

Department of Electronics and Computers

Study programme: Applied Electronics

Verification Project

Decimal-to-Gray Code Converter

Table of Contents

1	The digital design of the circuit	3
1.1	Block Diagram	3
1.2	Description of the general functionality of the DUT (Device Under Test)	3
1.3	DUT Interfaces	4
1.4	List of DUT Functionalities	6
1.5	Schematic Diagram	6
1.6	Waveforms	9
1.7	Register Table	9
2	Digital Circuit Verification	11
2.1	Verification Environment Architecture	11
2.1.1	SPI Interface Driver	12
2.1.2	APB Driver	12
2.1.3	Scoreboard / Reference Model	12
2.1.4	Monitor APB	13
2.1.5	SPI Interface Monitor	13
2.1.6	APB Interface Transaction	13
2.1.7	SPI Interface Transaction	13
2.1.8	SPI Interface Agent	13
2.1.9	APB Interface Agent	14
2.1.10	SPI Interface Coverage Collector	14
2.1.11	Scoreboard / Reference Model Coverage Collector	14
2.1.12	Coverage Collector for APB Interface	14
2.1.13	APB Sequences	14
2.2	Verification Checklist	15
2.2.1	Assertion Enable Mechanism	15
2.3	Functional Coverage Elements	16
2.4	Verification Scenarios and Tests	18
2.5	Results	20

1 THE DIGITAL DESIGN OF THE CIRCUIT

Block Diagram

General Functionality Description of the DUT (Device Under Test)

DUT Interfaces

List of DUT Functionalities

Schematic Diagram

Waveforms

Register Table

1.1 BLOCK DIAGRAM

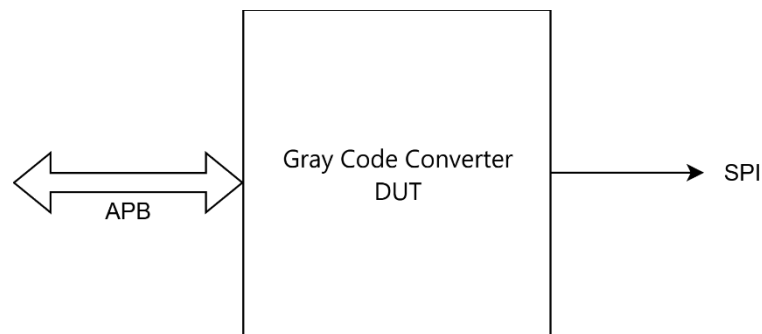


Figure 1. Block diagram of the converter

1.2 DESCRIPTION OF THE GENERAL FUNCTIONALITY OF THE DUT (DEVICE UNDER TEST)

The DUT converts an 8-bit binary number to Gray code. The input value (0–255) is received via the APB slave interface and stored in the OPERAND register. The Gray code conversion is performed using the formula: $\text{gray} = \text{binary} \text{ XOR } (\text{binary} \gg 1)$. The result is transmitted through the SPI master interface. At the input, we will enter a number transmitted through the APB interface, the DUT will contain the logic through which the respective number will be converted to Gray code using an XOR between the most significant bit and the bit shifted to the right, and at the output we will display the converted number through the SPI interface on 8 bits. Since the output will be on 8 bits, we will need to choose a number between 0 and 255, which represents the maximum number written on 8 bits. Since the input is an 8-bit value (0–255) and Gray code conversion preserves the bit width, no overflow is possible. The PSLVERR signal is asserted

when: (1) accessing an invalid/undefined address, or (2) attempting to write to the read-only RESULT register (address 0x02). Both the APB and SPI protocols will operate on the same clock.

1.3 DUT INTERFACES

Table 1. System signals generated in the testbench

General Signals			
Signal Name	I/O	Size	Description
clk	I	1	Clock signal
reset	I	1	Asynchronous reset, active low

Table 2. APB Interface

APB Interface			
Signal Name	Number of bits	Direction	Description
PCLK	1	Input	Clock signal
PRESET	1	Input	Reset signal, active LOW
PADDR	ADDR_WIDTH	Output	Address signal, up to 32 bits
PSELx	1	Output	Used for slave selection
PENABLE	1	Output	Output Indicates the second and subsequent transfer cycles
PWRITE	1	Output	Write access
PWDATA	DATA_WIDTH	Output	Write data bus, when PWRITE is HIGH
PREADY	1	Input	Slave can accept data and address transfers

PRDATA	DATA_WIDTH	Input	Read data bus, when PWRITE is LOW
PSLVERR	1	Input	Transfer errors

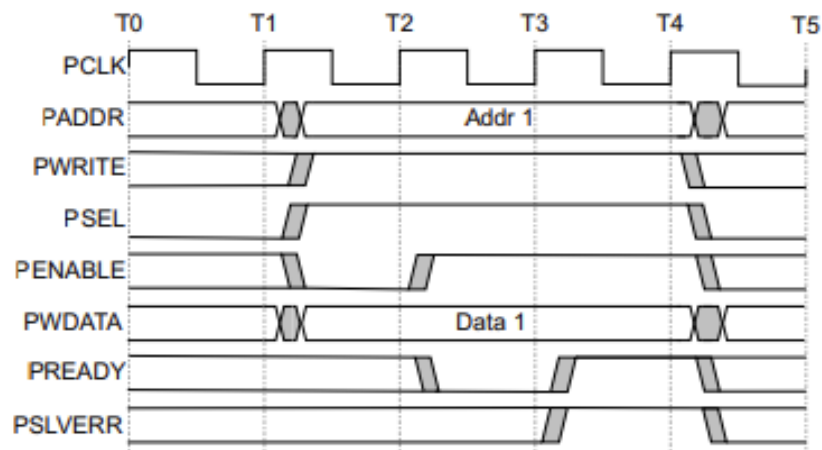


Figure 3-6 Example failing write transfer

Figure 2. APB protocol with write transfer fail

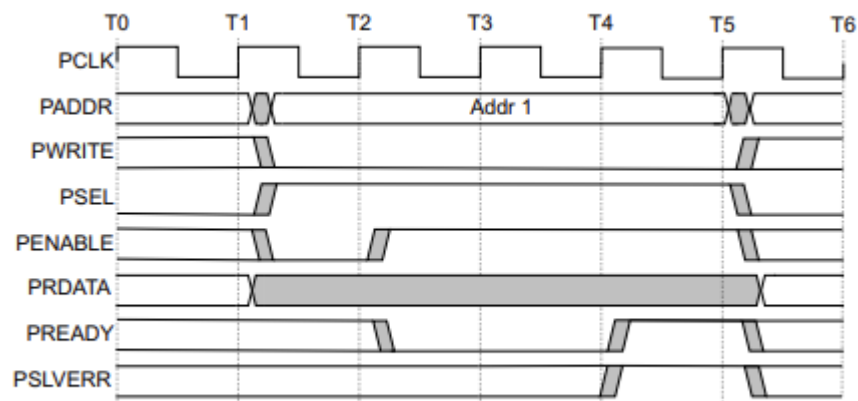


Figure 3-7 Example failing read transfer

Figure 3. APB protocol with read transfer fail

Table 3. Result display interface signals

SPI Result Display Interface			
Signal Name	Number of bits	Direction	Description

SCLK	1	Output	SPI clock signal
MOSI	1	Output	Data line for transmitting the converted number
MISO	1	Input	Data line for reception (not required for TX only)
CS	1	Output	Peripheral select (chip select), active on 0

1.4 LIST OF DUT FUNCTIONALITIES

- Reception of the input number, and storing it in a register
- Receiving an integer (base 10 with values between 0-255) through the APB interface.
- Conversion to Gray code.
- Transmitting the result through SPI....reg_config bit 0 set to 1.
- Sending the converted number through the SPI interface, using 8-bit packets.
- Notifying the user through an error signal transmitted on APB if an incorrect address was selected.
- Providing a reset signal to return to the initial state.
- Communication on the APB and SPI interfaces
- Compliance with APB communication protocols for input/output and SPI for output.
- It is mandatory to have at least one clock cycle between APB transactions
- Data on MOSI is transmitted on the rising edge of the SCLK signal after the reg_result register has reached 0.

1.5 SCHEMATIC DIAGRAM

At the input, we will receive a number in base 10, loaded into the OPERAND1 register. There, we will have the APB interface through which we will transmit the number to the DUT. The DUT, through its logic, has the role of transforming the number specified at the input and

transmitted through the APB protocol into a binary number, then that number will be encoded in Gray code. The number we want to encode will be written on pwdata, bit by bit, an APB-specific signal, then we will obtain the Gray code through an EXCLUSIVE OR (XOR) between the current bit and the bit shifted to the right. The obtained code will be transmitted at the output through the SPI protocol, subsequently stored in the RESULT register. Loading the result into the RESULT register will be done starting with the most significant bit. Transmitting the result at the output will be done through the SPI protocol, bit by bit, with the help of a counter that will decrement each time it transmits a bit. When it has reached 0, the control register will set the bit corresponding to the completion of the transmission.

Before performing the encoding, we must verify that the correct address was accessed for writing data. If this was not done, an error message will appear. If the correct address was written, we proceed with the encoding.

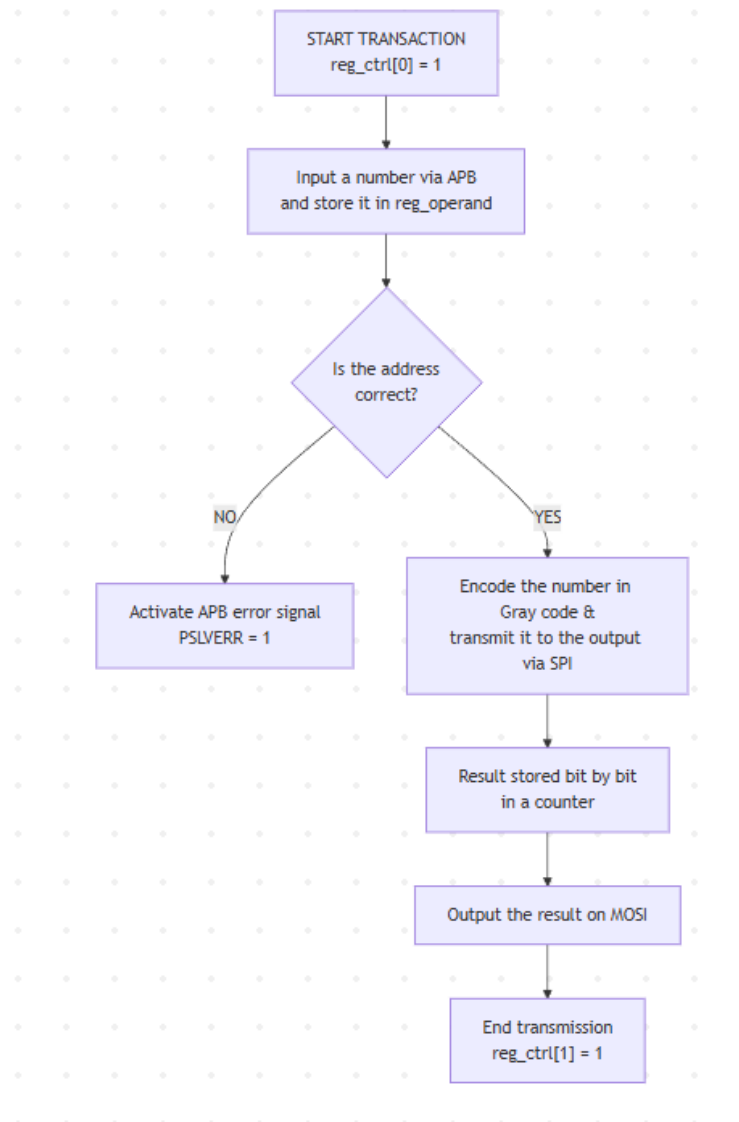


Figure 4: Logical schematic of the DUT

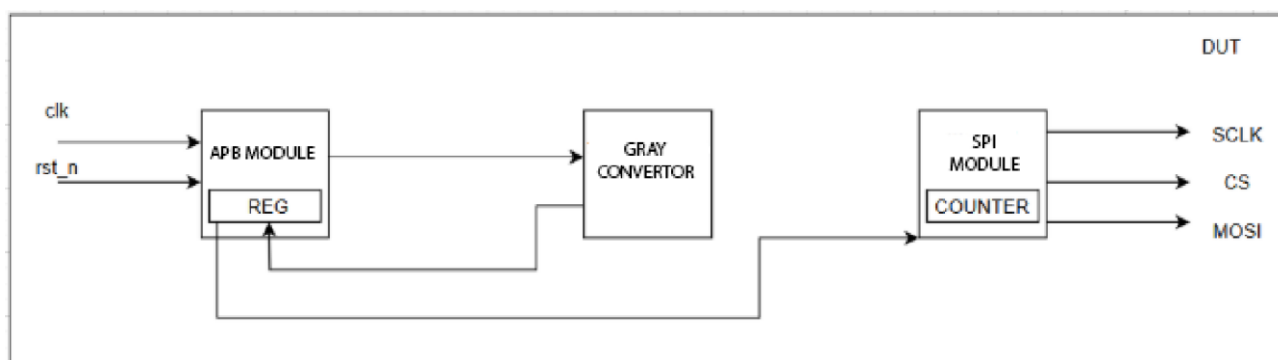


Figure 5: Block diagram of the DUT

Issues encountered:

- Data transmission on SPI cannot be done with the same clock through which we receive data on APB. According to the SPI protocol, SCLK operates only when CS is 0, meaning that is when transmission will begin. Thus, the 2 clocks will not synchronize. The CLK signal will be different from SCLK.
- The CS signal was not synchronizing with MOSI. This is very important because in the SPI protocol, to transmit data on MOSI, CS must be at 0. In experiments, MOSI appeared first, then CS. Solution: Shifting the CS signal by one clock cycle to the left (old_cs).

1.6 WAVEFORMS

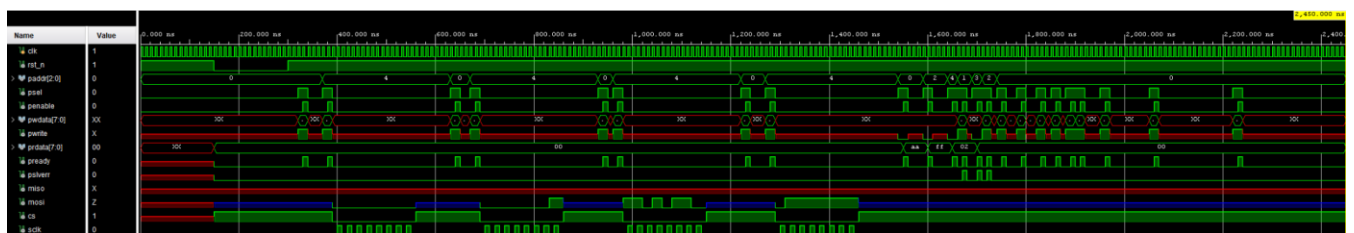


Figure 7: Waveforms from the DUT

1.7 REGISTER TABLE

A table will be created containing the register names, the addresses at which they can be accessed, as well as their descriptions.

Table 4. Registers accessible by the DUT

Register Name	Access Address	Register Structure	Register Field Description	Register Access Type
---------------	----------------	--------------------	----------------------------	----------------------

OPERAND	0x00	7 0 OPERAND	Operand value (in base 10)	write/ read
RESULT	0x02	7 0 RESULT	Result value (in Gray Code)	read
CTRL	0x04	7 6 2 1 0 Error Reserved End Start	-0 START Write 1 to begin SPI transmission. Auto-resets to 0 after one clock cycle (pulse behavior). -1 END Set by hardware when the SPI transmission completes successfully (all 8 bits transmitted). Cleared when a new START is written. - 6-2 Reserved bits (set to 0). - 7 Set by hardware when an SPI transmission was interrupted before all 8 bits were sent (CS deasserted prematurely).	write/ read

Parameters used:

- Parameter NO_OF_SPI_BITS = 8 => bits used for transmission in the SPI case

2 DIGITAL CIRCUIT VERIFICATION

Verification Environment Architecture

Verification Checklist

Functional Coverage Elements

Verification Scenarios

Tests Results

2.1 VERIFICATION ENVIRONMENT ARCHITECTURE

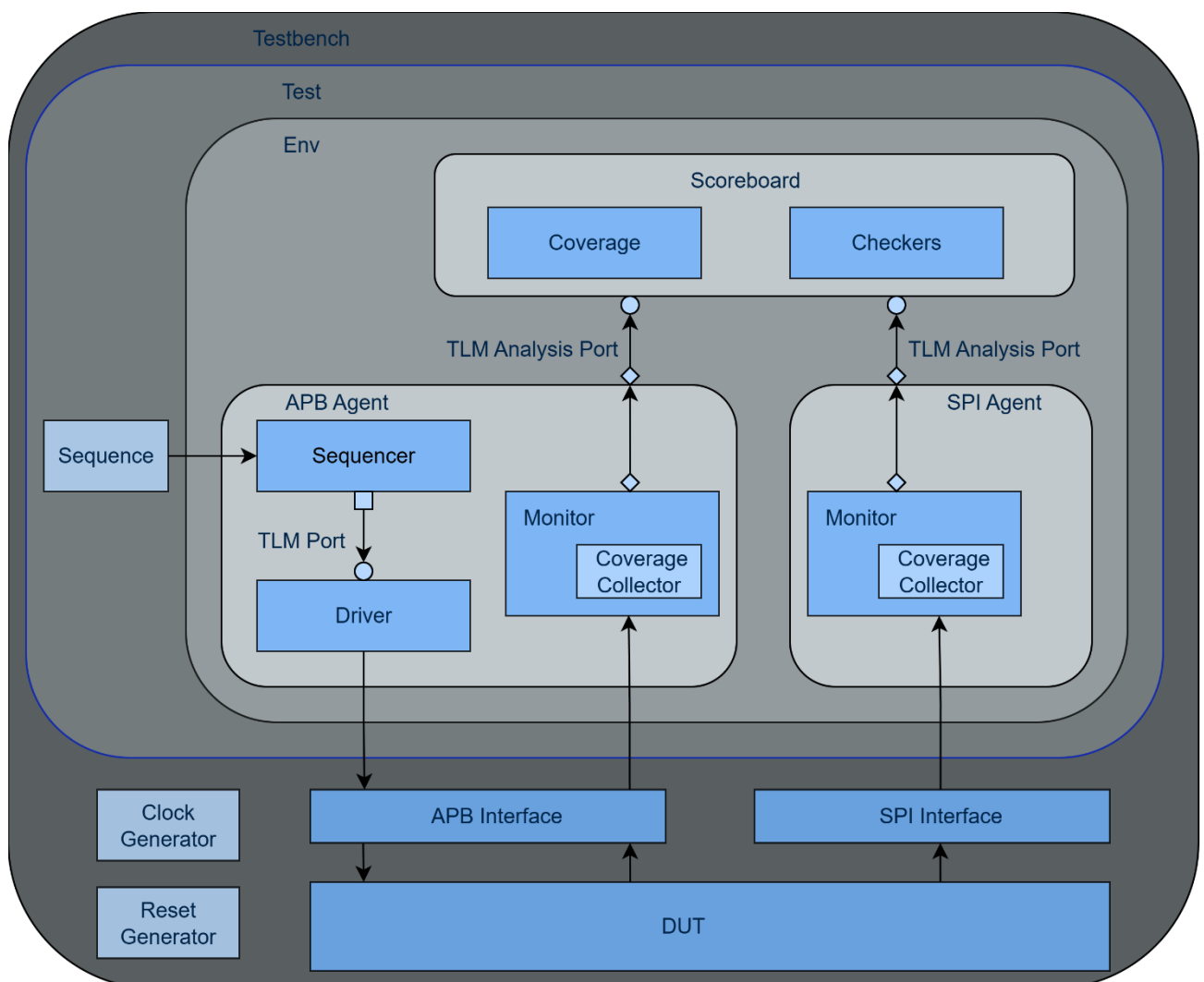


Figure 8. Overview of the components necessary for sending stimuli to the DUT via the APB interface

2.1.1 SPI Interface Driver

The SPI interface is an output-direction port of the DUT which is used to send the Gray code conversion result bit by bit on the MOSI line (the MISO line is declared in the interface but it is not used). This interface does not have a driver or sequencer as the data is generated by the DUT and consumed externally. The only way to provide stimulus for the conversion (writing the OPERAND register and asserting the START bit in CTRL) is via the APB Driver. SPI output is viewed using SPI Monitor.

2.1.2 APB Driver

The APB driver interacts directly with the DUT, sending transactions generated at an abstract level through the interface specified by the APB_DRIV_IF macro. These transactions follow the APB communication protocol and can include read or write operations to the DUT registers.

When the psel signal is active, the driver transmits the address, the transaction type (pwrite) and, in the case of a write, the pwrite value taken from the item. For read transactions, the value is taken from the prdata signal. According to the protocol specifications, the penable signal is asserted one clock cycle after psel.

2.1.3 Scoreboard / Reference Model

The scoreboard operates as a functional verification component for a digital module that performs the conversion of an integer to Gray code, using the APB and SPI interfaces. Its role is to verify the correctness of the DUT's operation by comparing the values received from SPI with the values calculated internally. Communication with the DUT is done through three registers accessed via APB:

- OPERAND (address 0x00): register that contains the input value (in base 10) for conversion. It can be read and written.
- RESULT (address 0x02): register that contains the result of the Gray code conversion. It is read-only.
- CTRL (address 0x04): controls the operation state. Writing 1 to the START bit (bit 0) triggers the conversion. The END bit (bit 1) indicates the completion of the operation. Bits 6–2 are reserved, and bit 7 (INVALID) indicates whether the last conversion was invalid.

The scoreboard receives transactions from APB, which it analyzes based on address. In the case of a write transaction, the values are stored in the corresponding registers and sampled in the functional coverage group. In the case of reads, they are compared with the already stored values to verify consistency. Subsequently, the operand value is converted locally to binary, then to Gray code. Transactions coming from SPI are then compared with the expected value (calculated by the scoreboard). If there are discrepancies, an error message is generated. Additionally, an inverse decoding from Gray to binary is performed to verify that the process is complete and correct.

2.1.4 Monitor APB

The APB monitor is a passive component that captures bus signals through the virtual interface and reconstructs APB transactions. It operates on the negative edge of PCLK to ensure stable signal sampling (the driver drives signals on the positive edge). For each transaction, the monitor:

- Counts the number of idle clock cycles (PSEL=0) between transactions, stored as the delay field
- Detects the SETUP phase when PSEL becomes 1
- Waits for the ACCESS phase completion (PENABLE=1 and PREADY=1)
- Captures address, write direction, data (PWDATA for writes, PRDATA for reads), and PSLVERR
- Publishes the transaction to the scoreboard via a TLM analysis port
- Triggers sampling of the APB coverage collector

2.1.5 SPI Interface Monitor

The SPI monitor is a passive component that captures the CS, SCLK and MOSI signals from the DUT. It detects the valid transmitted sequences and reconstructs the data packet. It sends the transaction to the scoreboard for validation. It operates only when CS is active (logic 0).

2.1.6 APB Interface Transaction

A transaction class in UVM represents a flexible and modular mechanism used to model, store and manipulate data and signals that circulate between the verification environment components. It facilitates communication between the generator, driver, monitor and other testbench components, allowing abstract management of transactions, as well as applying constraints and generating random values in accordance with the tested protocol.

2.1.7 SPI Interface Transaction

The SPI transaction defines the structure of the transmitted data: the value converted to Gray code, sent bit by bit. It contains a single 8-bit field (data) representing the reconstructed Gray code value from the MOSI line.

2.1.8 SPI Interface Agent

The SPI agent is a passive agent that contains only a monitor. Since the SPI interface is an output of the DUT, no driver or sequencer is needed. The monitor captures the data transmitted on the MOSI line and sends it to the scoreboard for verification.

2.1.9 APB Interface Agent

An agent encapsulates the Sequencer, Driver and Monitor into a single entity, connecting the components using the virtual interface.

2.1.10 SPI Interface Coverage Collector

The SPI coverage collector monitors the range of Gray code values transmitted on the MOSI line. The covergroup `stari_spi_cg` contains a single coverpoint on the transmitted data with 4 bins:

- `min_data`: value 0
- `random_data`: values 1–154
- `random_data_1`: values 155–254
- `max_data`: value 255

SPI protocol compliance (CS activation, SCLK behavior) is verified separately through assertions defined in the SPI interface module, not through the coverage collector.

2.1.11 Scoreboard / Reference Model Coverage Collector

The coverage collector in the scoreboard strictly monitors the values written on the APB bus, through the `registers_cg` group, sampled at each WRITE transaction. This guarantees that all extremes (0 and 255), the intermediate zones of the operand, and the status bits (INVALID, END) are exercised at least once. The Binary → Gray conversion emitted on SPI is not covered by a separate covergroup; its correctness is validated functionally by the scoreboard through direct comparison between the actual packet and the reference model prediction (`verify_data_match`). If the value does not match, a `uvm_error` is reported, otherwise the test is marked "PASS".

2.1.12 Coverage Collector for APB Interface

The APB coverage collector (`coverage_apb`) is instantiated inside the APB monitor and samples transaction data after each detected APB transfer. It contains the covergroup `stari_apb_cg` with 6 coverpoints: address (`addr_cp`), inter-transaction delay (`delay_cp`), error status (`perror_cp`), write direction (`pwrite_cp`), data value (`data_cp`), and a cross coverage of address x direction (`cross_addr_cp_pwrite_cp`). This collector achieves 100% coverage with the `test_coverage_100` test.

2.1.13 APB Sequences

The APB sequences generate stimulus for the APB driver. Each sequence creates one or more constrained-random transactions that the driver converts into APB bus cycles. The following sequences are implemented:

Sequence	Description
----------	-------------

apb_seq_write_addr0	Writes a random data value to the OPERAND register (address 0x0). Delay constrained to 0–3 cycles.
apb_seq_write_addr0 _fixed_data	Writes a specific (user-defined) data value to the OPERAND register. Used for targeted testing of known operands.
apb_seq_write_addr4	Writes the START command (0x01) to the CONTROL register (address 0x4) to initiate SPI transmission.
apb_seq_read	Reads from a specified address. Used to verify register contents after operations.
apb_seq_generic	Fully configurable sequence: allows specifying arbitrary address, write/read direction, data value, and inter-transaction delay. Used for targeted testing of invalid addresses, error conditions, and specific delay bins for coverage.
apb_sequence	General-purpose random sequence that generates 10–15 transactions with random addresses (0–4), random write/read direction, and delays constrained to 0–3 cycles.

2.2 VERIFICATION CHECKLIST

2.1 Assertion Enable Mechanism

All APB assertions use a disable iff guard that keeps them inactive until the reset signal (rst_n) has completed a full LOW-to-HIGH transition. This prevents false assertion failures during the initial simulation phase when DUT output signals (PREADY, PSLVERR) may be in unknown (X) states.

The mechanism uses two internal flags:

- rst_n_seen_low — set to 1 when rst_n goes LOW (negedge detected)
- rst_n_was_applied — set to 1 when rst_n returns HIGH after having been LOW

Assertions are only active when rst_n_was_applied == 1, i.e., the DUT has been through at least one complete reset cycle.

Table 5. SPI Interface Assertions

Assertion	Description
sclk_unmoved_on_idle_ss	Verifies that SCLK has no rising edges when CS is inactive (CS=1).
cs_deassert_clean	Verifies that when CS transitions from 0 to 1 (end of transfer), SCLK is 0.
mosi_valid_during_transfer	Verifies that MOSI is not X or Z during an active SPI transfer (CS=0).

Table 5.1 APB Interface Assertions

Assertion	Description
pready_is_pulse	Verifies that pready behaves as a pulse.
pready_fell_with_psel	Verifies that pready goes from one to zero when psel goes from 1 to 0.
psel_before_penable	Verifies that the psel signal is active one cycle before penable.
pwrite_during_transaction	Verifies that the pwrite signal remains active for the entire duration of the write transaction.
pready_unknown	Verifies that the pready signal is active only at the end of the transaction, when penable and psel are no longer active
pslverr_when_pready	Verifies that the pslverr signal is active only when pready becomes 1.
penable_single_cycle	Verifies that the penable signal is active for only a single cycle.
paddr_stable_during_access	Verifies that PADDR remains stable during the ACCESS phase (when both PSEL and PENABLE are active).

2.3 FUNCTIONAL COVERAGE ELEMENTS

Table 6. Functional coverage elements for APB/SPI data (scoreboard)

Event: registers_cg.sample(): The event is triggered at each APB write transaction.	
Cover Point	Description
reg_operand1_cp	The value of the operand received for conversion. Monitored: - Minimum value (0) - Values between 1 and 127 - Values between 128 and 254 - Maximum value (255)
reg_result_cp	4 automatic bins (random_val[4]) spanning values 1–254
invalid_cp	invalid_cp: Monitors bit 7 of reg_ctrl (the INVALID flag, which indicates if the last SPI transmission was incomplete).
end_cp	Monitors the completion of the conversion (bit 1 of reg_ctrl).

Table 7. APB Interface Coverage Elements (coverage_apb — stari_apb_cg)

Event: stari_apb_cg.sample() — triggered by the APB monitor after each captured transaction.	
Cover Point	Description
addr_cp	Address accessed. Bins: operand_reg (0x00), result_reg (0x02), control_reg (0x04), invalid_addresses (any other)
delay_cp	Number of idle clock cycles between transactions. Bins: minimal_delay (0–3), small_delay (4–8), large_delay (9–15), other_delays (16+)
perror_cp	PSLVERR status. Bins: no_error (0), error (1)
pwrite_cp	Transaction direction. Bins: read (0), write (1)
data_cp	Data value on the bus. Bins: zero (0), low_range (1–84), mid_range (85–169), hi_range (170–254), maximum (255)
cross_addr_cp_pwrite_cp	Cross coverage of address x direction (all combinations of addr_cp and pwrite_cp)

Table 8. SPI Interface Coverage Elements (coverage_spi — stari_spi_cg)

Event: stari_spi_cg.sample() — triggered by the SPI monitor after each captured SPI transaction.	
Cover Point	Description
data	Gray code value transmitted on MOSI. Bins: min_data (0), random_data (1–154), random_data_1 (155–254), max_data (255)

2.4 VERIFICATION SCENARIOS AND TESTS

Table 9. Verification Scenarios and Test Traceability

Scenario	Description
Register reset values	Read all registers immediately after reset to verify they are 0
Register write/readback	Write all writable registers with values, then read them back
Fast writes to OPERAND	Write to OPERAND with short delays (< 3 clock cycles) without waiting for conversion
SPI during conversion	Write operand and immediately start SPI transmission
Operand = 255	Write maximum value (255) to OPERAND register
Boundary values	Test converter with input values at coverage interval boundaries
Invalid APB write	Attempt write with protocol violation (PENABLE without PSEL)
Write to undefined address	Write to an address not in the register map (e.g., 0x01, 0x03)
Write registers out of order	Write CTRL before OPERAND
Reset during SPI transfer	Activate reset while SPI transfer is in progress

Scenario	Description
Register reset values	Read all registers immediately after reset to verify they are 0
Register write/readback	Write all writable registers with values, then read them back
Fast writes to OPERAND	Write to OPERAND with short delays (< 3 clock cycles) without waiting for conversion
SPI during conversion	Write operand and immediately start SPI transmission
Operand = 255	Write maximum value (255) to OPERAND register
Boundary values	Test converter with input values at coverage interval boundaries
Invalid APB write	Attempt write with protocol violation (PENABLE without PSEL)
SPI with unstable CS	CS toggles during transfer
Incomplete SPI transfer	Interrupt SPI after partial bit transfer

Table 10. List of implemented tests for DUT verification

Test file name	Test description
test_boundary_values	Tests Gray conversion for boundary values: 0x00, 0xFF, 0x01, 0x80, 0x55, 0xAA, 0x7F, 0xFE
test_register_readback	Writes an operand, then reads all 3 registers (OPERAND, RESULT, CONTROL) to verify readback correctness
test_write_readonly	Attempts to write to the RESULT register (read-only). Verifies PSLVERR is raised and RESULT is unchanged
test_invalid_addresses	Writes to invalid addresses (1, 3, 5, 6, 7). Verifies PSLVERR for each

s	
test_multiple_conversions	Executes 5 consecutive conversions without pause to verify back-to-back operation
test_end_bit	Performs a complete conversion and verifies the END bit (bit 1 of CONTROL) is set after SPI completes
test_random_conversions	Runs 10 conversions with random operands (0–255)
test_default_values	Reads all registers immediately after reset, verifying they are 0
test_full_flow	Complete end-to-end flow: write operand, start SPI, wait, read all registers back
test_operand_overwrite	Performs two consecutive conversions with different operands, verifying RESULT updates correctly
test_coverage_100	Systematically exercises all coverage bins across 5 phases: SPI values, register reads, invalid addresses, data ranges, and delay ranges. Achieves 100% APB and 100% SPI coverage

2.5 RESULTS

Running the test_coverage_100 test produced the following results:

- APB functional coverage: 100.00% (all 60 bins hit across 6 coverpoints + cross coverage)
- SPI functional coverage: 100.00% (all 4 bins hit)
- Scoreboard coverage (registers_cg): all bins for reg_operand1_cp, reg_result_cp, invalid_cp, and end_cp were exercised
- APB transactions captured: 23 out of 23
- SPI transactions verified: 4 out of 4 (all Gray code values matched predictions)

All assertions, 8 APB + 3 SPI, passed without failures throughout the simulation.